

秋招面试八股：斐波那契优化优化再优化

算法小小怪下士

2026 年 3 月 20 日

摘要

斐波那契数列是算法面试中的经典问题，其定义简单但优化空间巨大。本文从最朴素的递归出发，逐步讨论动态规划、矩阵快速幂、快速倍增法等优化技巧，并分析每种方法的时间复杂度和空间复杂度。

1 问题定义

斐波那契数列通常定义为：

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad (n \geq 2)$$

要求计算第 n 项的值。

2 朴素递归（直接递归）

最直观的方法，直接根据定义递归计算。

```
1 def fib(n):  
2     if n <= 1:  
3         return n  
4     return fib(n-1) + fib(n-2)
```

Listing 1 朴素递归

时间复杂度： $O(2^n)$ 。空间复杂度： $O(n)$ —— 递归栈深度。

3 带备忘录的递归（自顶向下动态规划）

使用一个字典（或数组）记录已计算过的值，避免重复。

```
1 def fib(n, memo={}):  
2     if n in memo:  
3         return memo[n]  
4     if n <= 1:  
5         return n  
6     memo[n] = fib(n-1, memo) + fib(n-2, memo)  
7     return memo[n]
```

Listing 2 备忘录递归

时间复杂度： $O(n)$ —— 每个子问题只计算一次。空间复杂度： $O(n)$ —— 存储备忘录 + 递归栈。

4 动态规划（自底向上，数组版）

用数组保存所有子问题的解，从小到大递推。

```
1 def fib(n):  
2     if n <= 1:  
3         return n
```

```

4     dp = [0] * (n+1)
5     dp[1] = 1
6     for i in range(2, n+1):
7         dp[i] = dp[i-1] + dp[i-2]
8     return dp[n]

```

Listing 3 DP 数组版

时间复杂度: $O(n)$, 空间复杂度: $O(n)$

5 重点: 动态规划 (自底向上, 滚动变量)

仅保留前两个状态, 空间压缩至常数。

```

1 def fib(n):
2     if n <= 1:
3         return n
4     a, b = 0, 1
5     for _ in range(2, n+1):
6         a, b = b, a + b
7     return b

```

Listing 4 滚动变量 DP

时间复杂度: $O(n)$ 空间复杂度: $O(1)$ 这是最常用的线性解法, 适合 n 在百万级以内。

6 重点: 矩阵快速幂

利用矩阵恒等式:

$$\begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F(1) \\ F(0) \end{bmatrix}$$

通过快速幂计算矩阵的幂, 时间复杂度降至对数级。

```

1 def fib(n):
2     if n <= 1:
3         return n
4     def mat_mul(A, B):
5         return [[A[0][0]*B[0][0]+A[0][1]*B[1][0],
6                 A[0][0]*B[0][1]+A[0][1]*B[1][1],
7                 A[1][0]*B[0][0]+A[1][1]*B[1][0],
8                 A[1][0]*B[0][1]+A[1][1]*B[1][1]]
9     def mat_pow(mat, k):
10        result = [[1, 0], [0, 1]]
11        while k:
12            if k & 1:
13                result = mat_mul(result, mat)
14                mat = mat_mul(mat, mat)
15                k >>= 1
16        return result
17    M = [[1, 1], [1, 0]]
18    Mn = mat_pow(M, n-1)
19    return Mn[0][0]

```

Listing 5 矩阵快速幂

时间复杂度: $O(\log n)$, 矩阵乘法可以视为常数。空间复杂度: $O(1)$ 。适合 n 极大 (如 10^{18}) 且需要精确值的场景。

7 快速倍增法

利用恒等式【可以由归纳法证得】:

$$F(2k) = F(k)[2F(k+1) - F(k)]$$

$$F(2k+1) = F(k+1)^2 + F(k)^2$$

通过递归分治计算，常数比矩阵法更小。

```

1 def fib(n):
2     if n == 0:
3         return (0, 1)    # (F(n), F(n+1))
4     else:
5         a, b = fib(n >> 1)    # a = F(k), b = F(k+1)
6         c = a * (2 * b - a)    # F(2k)
7         d = a * a + b * b      # F(2k+1)
8         if n & 1:
9             return (d, c + d)
10        else:
11            return (c, d)

```

Listing 6 快速倍增法

时间复杂度: $O(\log n)$, 空间复杂度: $O(\log n)$ 。适合大数计算。

8 通项公式

$$F(n) = \frac{\phi^n - \psi^n}{\sqrt{5}}, \quad \phi = \frac{1 + \sqrt{5}}{2}, \quad \psi = \frac{1 - \sqrt{5}}{2}$$

```

1 def fib(n):
2     phi = (1 + 5**0.5) / 2
3     psi = (1 - 5**0.5) / 2
4     return round((phi**n - psi**n) / 5**0.5)

```

Listing 7 通项公式

时间复杂度: 浮点运算可视为 $O(1)$, 但指数运算实际为 $O(\log n)$, 【快速幂算法的复杂度】。空间复杂度: $O(1)$

9 总结对比

表 1 各方法复杂度对比

方法	时间复杂度	空间复杂度	适用场景
朴素递归	$O(2^n)$	$O(n)$	直观应用定义, n 极小
备忘录递归	$O(n)$	$O(n)$	简单实现, n 较小
DP 数组版	$O(n)$	$O(n)$	需要保存中间结果
滚动变量 DP 【重点】	$O(n)$	$O(1)$	n 在百万级以内, 常用
矩阵快速幂 【重点】	$O(\log n)$	$O(1)$	n 极大 (如 10^{18}), 精确值
快速倍增法	$O(\log n)$	$O(\log n)$	n 极大, 常数更优
通项公式	浮点 $O(1)^*$	$O(1)$	n 很小, 浮点近似